

Tool Calling in Python — Beginner Tutorial

A practical tutorial using the OpenAI-compatible Chat Completions API with Groq.

1. The basic idea

Tool calling = the model decides **WHAT** function to call; your Python code decides **HOW** that function actually runs.

The complete flow is: User asks a question → model chooses a tool → model returns a tool call → Python parses the call → Python executes the real function → Python sends the result back → model produces the final answer.

2. Function vs tool vs tool calling

Function: a normal Python function such as `get_time(city)`.

Tool: a function exposed to the model with a schema describing its name, purpose, and parameters.

Tool calling: the model chooses a tool and asks your application to execute it. The model does not directly execute your Python function.

3. The important Chat Completions fields

- `assistant_message.tool_calls` — contains the tool calls requested by the model.
- `tool_call.id` — the unique ID for that tool call.
- `tool_call.function.name` — the name of the selected tool.
- `tool_call.function.arguments` — arguments returned by the model as a JSON string.
- `tool_call_id` — the same ID sent back when returning the tool result.

4. What happens for the Karachi example?

For `What time is it in Karachi?`, the model may return a tool call conceptually like this:

```
tool_call.id
  ↓
fc_123...

tool_call.function.name
  ↓
get_time

tool_call.function.arguments
  ↓
{"city": "Karachi"}
```

Your code then looks up `get_time`, parses the arguments, and executes `get_time(city='Karachi')`.

The result is then returned with `role='tool'` and the same `tool_call_id`. This links the result to the model's original request.

5. Why `json.loads()` is needed

The SDK gives tool arguments as a JSON string, for example `'{"city": "Karachi"}'`. `json.loads()` converts that string into a Python dictionary so it can be passed to the function.

```
arguments = json.loads(tool_call.function.arguments)

# Result:
{"city": "Karachi"}
```

6. Why ****arguments** is used

If the dictionary is {"city": "Karachi"}, then function(**arguments) is equivalent to function(city="Karachi"). For the calculator, {"a":25, "b":75} becomes calculate(a=25, b=75).

7. Why the tool registry is useful

```
tool_functions = {
    "get_weather": get_weather,
    "get_time": get_time,
    "calculate": calculate,
}

function = tool_functions[tool_name]
```

This avoids a long chain of if/elif statements. The model gives the tool name, and the dictionary finds the matching Python function.

8. Why the conversation is sent again

After the first model response, the conversation needs to contain the assistant's tool request and the tool result. The second model request then has enough context to produce the final natural-language answer.

9. Complete commented code

```
from openai import OpenAI
import json
import os

# -----
# 1. Create the Groq client
# -----
# The OpenAI Python SDK can also talk to Groq because Groq
# provides an OpenAI-compatible API.
#
# api_key:
#   Your Groq API key.
#
# base_url:
#   Tells the SDK to send requests to Groq instead of OpenAI.

client = OpenAI(
    api_key="test key",
    base_url="https://api.groq.com/openai/v1",
)

# -----
# 2. Create our actual Python tools
# -----
# These are normal Python functions.
# The AI cannot directly execute these functions.
# Our application will execute them after the AI chooses a tool.

def get_weather(city: str):
    # Return a dummy weather result.
    return f"The weather in {city} is sunny and 30°C."

def get_time(city: str):
    # Return a dummy time result.
    return f"The current time in {city} is 3:30 PM."

def calculate(a: float, b: float):
    # Add two numbers and return the result.
    return a + b

# -----
# 3. Describe the tools to the AI
# -----
# The model does not automatically know about our Python functions.
# We give the model a "tool schema" that explains:
#
# - what the tool is called
```

```

# - what the tool does
# - what arguments it accepts
# - which arguments are required
#
# IMPORTANT:
# This does NOT execute the functions.
# It only tells the model which tools are available.

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get the weather for a city.",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {
                        "type": "string",
                        "description": "Name of the city"
                    }
                }
            },
            "required": ["city"],
        },
    },
    {
        "type": "function",
        "function": {
            "name": "get_time",
            "description": "Get the current time for a city.",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {
                        "type": "string",
                        "description": "Name of the city"
                    }
                }
            },
            "required": ["city"],
        },
    },
    {
        "type": "function",
        "function": {
            "name": "calculate",
            "description": "Add two numbers together.",
            "parameters": {
                "type": "object",
                "properties": {
                    "a": {"type": "number"},
                    "b": {"type": "number"}
                },
                "required": ["a", "b"],
            },
        },
    },
]

# -----
# 4. Connect tool names to Python functions
# -----
# The AI will return a tool NAME such as:
#
#     "get_time"
#
# We need to connect that name to the real Python function.
#
# This dictionary gives us:
#
#     "get_time"      -> get_time()
#     "get_weather"   -> get_weather()
#     "calculate"     -> calculate()

tool_functions = {
    "get_weather": get_weather,
    "get_time": get_time,
    "calculate": calculate,
}

# -----
# 5. Create the user's message
# -----
# This is the question we want the model to answer.

```

```

#
# Try changing this to:
#
# "What is 25 + 75?"
#
# Or:
#
# "What is 25 + 75? Also, what is the weather in New York
# and the current time in London?"

user_message = "What time is it in Karachi?"

# messages is the conversation history that we send to the model.
messages = [
    {"role": "user", "content": user_message}
]

# -----
# 6. Ask the model what to do
# -----
# We send:
#
# - the model name
# - the conversation
# - the available tools
#
# The model decides whether it can answer directly or needs a tool.
#
# IMPORTANT:
# The model does NOT execute get_time().
# It only REQUESTS that our application execute it.

response = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=messages,
    tools=tools
)

print("Model response:")
print(response)

print("Model message:")
print(response.choices[0].message)

# -----
# 7. Check whether the model requested a tool
# -----
# response.choices[0].message is the assistant's message.
#
# If tool_calls exists, the model wants our application
# to execute one or more tools.

assistant_message = response.choices[0].message

if assistant_message.tool_calls:

    # Add the assistant's tool-call message to the conversation.
    # This is important because the next model request needs
    # to know what tool the model previously requested.
    messages.append(assistant_message)

    # A model can request multiple tools.
    # Therefore, we loop through every tool call.
    for tool_call in assistant_message.tool_calls:

        # -----
        # 7A. Get the selected tool name
        # -----
        # Example:
        #
        #     "get_time"
        #
        tool_name = tool_call.function.name

        # -----
        # 7B. Get and parse tool arguments
        # -----
        # The model gives arguments as a JSON string.
        #
        # Example:
        #
        #     '{"city": "Karachi"}'
        #
        # json.loads() converts that string into a Python dictionary:
        #

```

```

#     {"city": "Karachi"}

arguments = json.loads(tool_call.function.arguments)

print("AI selected tool:", tool_name)
print("Arguments:", arguments)

# -----
# 7C. Find the real Python function
# -----
# Use the tool name returned by the model
# to find the matching Python function.

function = tool_functions[tool_name]

# -----
# 7D. Execute the Python function
# -----
# **arguments expands the dictionary.
#
# Example:
#
#     {"city": "Karachi"}
#
# becomes:
#
#     get_time(city="Karachi")

result = function(**arguments)

print("Tool result:", result)

# -----
# 7E. Send the tool result to the model
# -----
# The model requested a tool using tool_call.id.
#
# We send that SAME ID back as tool_call_id.
#
# This tells the model:
#
# "This result belongs to the tool call you made
# with this ID."

messages.append(
    {
        "role": "tool",
        "tool_call_id": tool_call.id,
        "content": str(result)
    }
)

# -----
# 8. Ask the model for the final answer
# -----
# Now the conversation contains:
#
# 1. User question
# 2. Model's tool request
# 3. Actual tool result
#
# The model can now use the tool result
# to create a normal human-readable answer.

final_response = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=messages
)

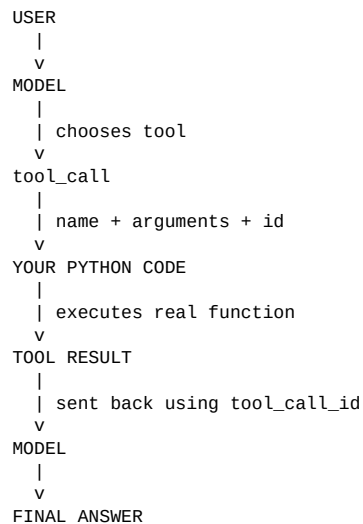
print("Final answer:")
print(final_response.choices[0].message.content)

else:

# If the model did not request a tool,
# it already has a normal answer.
print("Final answer:")
print(assistant_message.content)

```

10. Mental model to remember



Key rule

The model chooses the tool. Your application executes the tool. The tool result goes back to the model. The model then writes the final answer.

For Chat Completions: use `tool_call.id` and return it as `tool_call_id` in the tool message.